קונטיינרים בשפת הקודש

ערכה חן ג. , כל המידע וחלק מהקטעי קוד לקוחים מהחוברת קונטיינרים באנגלית.

קרדיטים **ענקיים** לרובי, איתמר צבי וגל נדג'ר הדבאופס.

Kill() – (כן, מתחילים אופטימי)

הורגת תהליך בעל PID **לוקלי** שמובא לKILL כפרמטר. הPID הוא **לוקלי** בקונטיינר, ולא גלובלי. למשל אם נריץ kill(1) בתוך קונטיינר אנחנו נהרוג את התהליך INIT שבקונטיינר **ולא** את התהליך INIT הכללי. אם הרגנו תהליך שהוא אבא של תהליכים אחרים, הם ישודכו לתהליך בעל PID 1 (INIT) של הקונטיינר. אם הרגנו את התהליך עם PID 1 בקונטיינר אז **כל** התהליכים בקונטיינר ימותו.

מה יוחזר?

בהצלחה 0.

בכישלון 1-.

Fork() – יוצר עותק של התהליך. התהליך החדש יהיה הבן של התהליך שקרא לfork והוא יקבל PID **גלובלי** חדש וייחודי, וגם PID **לוקלי** שייחודי בקונטיינר שהוא נמצא בו (אבל יכול להיות תהליך אחר **בקונטיינר אחר** עם אותו PID לוקלי).

מה יוחזר?

עבור האבא: הPID הגלובלי של הבן (כלומר הPID של הבן בNS של האבא).

עבור הבן: 0

בכישלון יוחזר 1-.

Getpid()

מה יוחזר?

הPID הלוקלי של התהליך.

בכישלון 1-.

unshare(PID_NS)

הולך ביחד עם fork

מה יוחזר?

0 עבור הצלחה, 1- עבור כישלון.

ישפיע רק על הPID של הבן (שנוצר לאחר unshare) של התהליך שהפעיל את הפקודה.

הבן יהיה בPID_NS משלו.

עבור קטע הקוד הבא, מה יודפס בהנחה שהPID של האבא הוא 3?

```
if (unshare(PID_NS) != 0) {
  printf(stderr, "Cannot create pid namespace\n");
  exit(1);
}
pid = fork();
if (pid == -1) {
  printf(stderr, "FAILURE: fork\n");
  exit(1);
}
if (pid == 0)
  printf(stdout, "New namespace. PID = %d\n", getpid());
else
  printf(stdout, "Parent's perspective on the child. PID = %d\n", pid);
```

יודפס:

New namespace. PID = 1

Parent's perspective on the child. PID = 4

האם בהכרח בסדר הזה?

לא. לא ידוע איזה מהתהליכים, האבא או הבן, ירוץ ראשון.

הסבר למה שהתרחש:

מבצעים unshare . בהנחה שהוא הצליח, מבצעים גם fork.

מכאן ואילך:

אם fork נכשל, יוחזר 1- והתוכנית תסתיים.

עבור הבן יוחזר 0 ולכן pid=0.

נבצע getpid שמחזיר pid לוקלי כלומר 1 כי הבן הוא התהליך הראשון ב namespace החדש בנוצר בעקבות הunshare .

עבור האבא יוחזר הpid של הבן בקונטיינר של האבא, מכיוון ש הpid של האבא בקונטיינר שלו הוא 3, הpid של הבן יהיה 4.

Mount()

בשביל מה זה טוב?

בעצם, כל הקבצים של המערכת יכולים להיות מפוזרים על פני כמה מכשירים מחוברים. פקודת MOUNT נועדה לחבר מערכת קבצים שנמצאת על אחד המכשירים (למשל USB) לעץ הקבצים הגדול. קיימת גם פקודת ()UMOUNT שכצפוי מנתקת את המערכת הזו.

מה mount עושה בכלל? (קרדיט לאיתמר וגל)

בעולם האמיתי, mount נועד להקל על השימוש בקבצים בכך שאנחנו יוצרים קישור למערכת קבצים כלשהי בתוך מערכת הקבצים הנוכחית שלנו (וכמובן רק בתוך ה-MOUNT_NS שלנו). כשאנחנו מחברים מכשיר USB למחשב, נרצה לגשת אליו בעזרת נתיב פשוט ואנושי שאנחנו מכירים ויודעים לעבוד איתו. כמו למשל, במקרה של לינוקס, /mnt/usb-1. אבל הוא לא באמת חלק ממערכת הקבצים, בשביל לגשת אליו, נצטרך לעשות mount למערכת הקבצים שנמצאת במכשיר ה-USB אל /mnt/usb-1

עוד דוגמא: אם יש תיקיית רשת במיקום nfs-server://var/shared, ואנחנו רוצים לגשת לתיקייה הזאת במערכת שלנו בנתיב /nfs, נוכל לבצע mount של nfs-server://var/shared על /nfs, ואז הנתיב /nfs בעצם יוביל אותנו אל nfs-server://var/shared

unshare(MOUNT_NS)

הולך ביחד עם mount

מה יוחזר?

0 עבור הצלחה, 1- עבור כישלון.

ישפיע על התהליך שמפעיל ישירות את הפקודה, אחרי unshare ואז mount יווצר עבורו mountpoint וכל הקבצים שיווצרו שם לא יהיו גלויים לתהליכים אחרים, מלבד הילדים שהוא ייצור לאחר מכן שלמעשה יירשו את הMOUNT_NS שלו.

internal_fs – מתייחס למכשיר עם מערכת קבצים עליו שמחברים למחשב – למשל: USB

מה יודפס עבור הקטע הבא?

```
int pid = fork();
if (pid == 0)
{
    unshare(MOUNT_NS);
    mkdir("dirA");
    mount("internal_fs_a", "dirA", 0)
    createNwrite("dirA/file.txt", "123456789\n", 10);
}

if (pid > 0)
{
    // make sure child process runs first to create a new ns
    sleep(10000);
    // create a mount point and mount on it a preformatted internal_fs_b
    mkdir("dirB");
    mount("internal_fs_b", "dirB", 0);
    createNwrite("dirB/file.txt", "987654321\n", 10);
}
```

```c
if(pid == 0){
    if (open("dirA/file.txt", O_RDONLY) >= 0)
    {
        printf(stderr, "son opened A\n");
        exit(1);
    }
    if (open("dirB/file.txt", O_RDONLY) >= 0)
    {
        printf(stderr, "son opened B\n");
        exit(1);
    }
}
else{
    if (open("dirB/file.txt", O_RDONLY) >= 0)
    {
        printf(stderr, "parent opened B\n");
        exit(1);
    }
    if (open("dirA/file.txt", O_RDONLY) >= 0)
    {
        printf(stderr, "parent opened A\n");
        exit(1);
    }
}
exit(0);
```

:יודפס

Son opened A

Parent opened B

למה? כי הם ב mount_ns נפרדים!

בגלל הunshare בהתחלה לאחר הfork, לכל אחד מהתהליכים יש mount_ns משלו. אז אין להם גישה לקבצים שצורפו בעזרת mount למערכת הקבצים של הNS השני.

Global mount namespace	New mount namespace
Mounted device: internal_fs_b; Mount point: dirB The root file system contains an empty dirA	Mounted device: internal_fs_a; Mount point: dirA The root file system contains an empty dirB

נקודה חשובה: לתהליכים יש גישה לתיקיות שנוצרו בNS השני, אבל מבחינתם הן ריקות!

עוד נקודה חשובה:

UNSHARE(MOUNT_NS) משפיע אך ורק אם יש MOUNT אחריו!

תרגילים ממבחנים

2023b 61

שאלה 6

הסבירו היטב את השפעת הפקודה unshare(MOUNT_NS) במערכת XV6 שתומכת בקונטיינרים.

במיוחד התייחסו לתפקיד פקודת mount בתהליך ההפרדה וציינו האם ההפרדה מתייחסת למערכת קבצים כולה.

תשובה והסבר:

הפקודה unshare(MOUNT_NS) יוצרת עבור התהליך שקורא לה mount namespace משלו, נפרד מזה של שאר התהליכים. לאחר הקריאה ל unshare(MOUNT_NS) צריך לקרוא לפקודה mount ובכך לחבר מערכת קבצים חיצונית **שתהיה גלויה רק לתהליך הזה ולבנים שהוא ייצור לאחר הפקודה UNSHARE .**

ההפרדה לא מתייחסת למערכת הקבצים כולה אלא רק לMountpoints.

בתרגילים הבאים צריך להגיד מה יודפס.

2023a 91

```
int pid = fork();
if (pid == 0) {
   mkdir (“dirS”);
   unshare (MOUNT_NS);
   mount (“internal_fs_a”,”dirS”,0); //mount a preformatted internal_fs_a
   int fd = open (“dirS/file.txt”,O_CREATE|O_RDWR);
   write (fd ,“123456789\n”,10);
   close (fd);
  }
if (pid > 0) sleep (10000); // wait for child to finish
sleep (10000); // both processes will sleep
if( pid == 0) {
    if (open (“dirS/file.txt”,O_RDONLY) < 0) printf (stderr, "son failed to open\n");
    else printf (stderr, "son succeeded\n");
  }
If (pid>0) {
   if (open (“dirS/file.txt”,O_RDONLY) < 0) printf (stderr, "father failed to open\n");
   else printf (stderr, "father succeeded\n");
 }
```

יודפס:

Son succeesed

Father failed to open

הסבר:

לאחר הfork הבן (ולמען האמת זה לא משנה אם זה הבן או האבא) עשה unshare ובכך הפריד את עצמו מmount_ns הגלובלי! לכן הקובץ שהוא יצר בתיקייה S לא גלוי לאבא.

2023a 92

```
unshare (MOUNT_NS);
int pid = fork();
if (pid == 0) {
   mkdir (“dirS”);
   mount (“internal_fs_a”,”dirS”,0); //mount a preformatted internal_fs_a
   int fd = open (“dirS/file.txt”,O_CREATE|O_RDWR);
   write (fd ,“123456789\n”,10);
   close (fd);
  }
if (pid > 0) sleep (10000); // wait for child to finish
sleep (10000); // both processes will sleep
if( pid == 0) {
   if (open (“dirS/file.txt”,O_RDONLY) < 0) printf (stderr, "son failed to open\n");
   else printf (stderr, "son succeeded\n");
  }
If (pid>0) {
   if (open (“dirS/file.txt”,O_RDONLY) < 0) printf (stderr, "father failed to open\n");
   else printf (stderr, "father succeeded\n");
 }
```

יודפס:

Son succeeded

Father succeeded

הסבר:

לאחר הUNSHARE האבא מקבל mount_ns משלו, ומוריש אותו גם לבן שהוא יוצר בפקודת FORK **שלאחר הUNSHARE**. לכן לשניהם תהיה גישה לקובץ שהבן ייצור בS לאחר הMOUNT.

```
int pid = fork();
if (pid == 0) {
  mkdir ("dirS");
  mount ("internal_fs_a","dirS",0); //mount a preformatted internal_fs_a
  int fd = open ("dirS/file.txt",O_CREATE|O_RDWR);
  write (fd ,"123456789\n",10);
  close (fd);
 }
if (pid > 0) {
sleep (10000); // wait for child to finish
unshare (MOUNT_NS);
 }
sleep (10000); // both processes will sleep
if( pid == 0) {
   if (open ("dirS/file.txt",O_RDONLY) < 0) printf (stderr, "son failed to open\n");
   else printf (stderr, "son succeeded\n");
 }
If (pid>0) {
   if (open ("dirS/file.txt",O_RDONLY) < 0) printf (stderr, "father failed to open\n");
   else printf (stderr, "father succeeded\n");
 }
```

יודפס:

Son succeeded

Father succeeded

הסבר:

הבן והאבא היו בהתחלה באותו namespace גלובלי, הבן עשה MOUNT **אבל בלי UNSHARE** קודם ולכן לא נוצר מרחב מבודד. הקבצים שהוא יוצר גלויים לכולם. אפילו שהאבא עשה UNSHARE אחר כך, זה לא משפיע משום שהוא עדיין רואה את מה שהיה במערכת קבצים קודם. **UNSHARE(MOUNT_NS) משפיע אך ורק אם יש MOUNT אחריו!**

מה יודפס, בהנחה ש getppid מחזירה את PID של האבא של התהליך?

```
unshare(PID_NS);
int pid = fork();
if(pid == 0) {
  pid = fork();
  if(pid == 0) {
   sleep(999); // long sleep
   kill(getppid());
   printf(stdout, "GrandSon PID=%d \n", getpid());
    }
  else {
    sleep(9999); // very long sleep
    printf(stdout, "Son PID=%d \n", getpid());
     }
  }
else {
  sleep(99999); //very very sleep enough for creating and running all son processes
  printf(stdout, "Father still alive \n");
 }
```

525 שאלון 89.7

יודפס:

Father still alive

הסבר:

הרצנו unshare(PIS_NS) ומיד אחריו FORK: קיבלנו שהבן נמצא בpid namespace חדש וPID שלו הוא 1. אחרי זה הורץ עוד FORK, ונוצר נכד. הנכד נמצא גם כן בקונטיינר החדש של הבן, וכשהוא מריץ KILL על 1 הוא הורג את **האבא שלו בקונטיינר** - שהוא הבן בעצם. **אבל** הבן הוא תהליך INIT בקונטיינר (כלומר תהליך מספר 1) ולכן אם הוא מת, כל מי שבקונטיינר מת איתו 🙁 . (כן כן – KILL היא לוקאלית בקונטיינר, FORK היא גלובלית. חשוב!!)

לכן הבן והנכד לא ידפיסו. רק האבא ידפיס.

אתה כמו תהליך init
בקונטיינר שלי



אני לא יכולה לחיות
בלעדייך

2022b 95

```
unshare(PID_NS);
int pid = fork();
if(pid == 0){
  pid = fork();
  if(pid == 0) {.
    sleep(99999); // very long sleep
    printf(stdout, "GrandSon PID=%d \n", getpid());
   }
  else {
    sleep(99999); // very long sleep
    printf(stdout, "Son PID=%d \n", getpid());
   }
 }
else {
  sleep(9999); // sleep enough for creating all son processes, but they are still sleeping
  kill(pid+1);
}
```

יודפס:

Son PID = 1

הסבר:

עושים UNSHARE ולכן הבן והנכד יהיו בקונטיינר חדש. האבא ישן הכי פחות זמן, ולכן הוא ירוץ ראשון ויריץ את הפקודה kill(pid+1). כידוע fork מחזירה את הPID של הבן, ולכן PID+1 יהיה הPID הגלובלי של ה**נכד**. אז האבא יהרוג את הנכד, ולכן לאחר מכן רק הבן ירוץ וידפיס את הערך שמוחזר מgetpid – הPID הלוקלי שלו בקונטיינר החדש כלומר 1.

2022a 86

ידוע שהPID הגדול ביותר לפני הפקודות הנ"ל היה 15. מה יודפס?

```
unshare(PID_NS);
pid = fork();
if (pid > 0)  printf(stdout, "Grandparent's perspective on the child. PID=%d \n", pid);
else {
   pid = fork();
   if (pid == 0) printf (stdout, "Son PID=%d \n", getpid());
   else printf (stdout, "Parent's perspective on the child. PID=%d \n", pid);
}
```

5
(5)

תשובה והסבר:

יודפס:

Grandparent's perspective on the child. PID = 16

Son PID = 2

Parent's perspective on the child. PID = **17**

הסבר:

הPID הגלובלי של הבן הוא 16, ולכן זה מה שהאבא ידפיס.

הבן יוצר נכד, והנכד נוצר בקונטיינר החדש ולכן getpid יחזיר 2.

עכשיו, fork בעיקרון מחזיר גלובלי, בשביל שהוא יחזיר לוקלי צריך לשנות אותו. **אבל** בקורס הזה אנחנו עובדים בXV6 ללא תמיכה בקונטיינרים מקוננים, ולכן fork לא תוכל לעבור הסבה לפונקציה שמחזירה לוקלי (פרטים נוספים בחוברת קונטיינרים כי זה גדול עליי).

בקיצור Fork תחזיר **גלובלי** ולכן יודפס 17 בסוף.